Lab 4: Các kiểu dữ liệu nâng cao trong C và thuật toán sắp xếp (Advanced C Types & Sorting)

Hướng dẫn thực hành Lập trình hệ thống

Trường Đại học Công nghệ - ĐHQGHN

Ngày 9 tháng 10 năm 2026

Mục lục

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Mục tiêu buổi thực hành

Sau buổi hôm nay, sinh viên phải làm được

1. **Mô hình hoá dữ liệu bằng kiểu tự định nghĩa:** `struct`, `typedef`, `enum`, `union` (có nhãn); *giải thích* được `sizeof` bằng căn lề và đệm, kiểm chứng bằng `offsetof`.
2. **Dùng con trỏ tới struct đúng mục đích:** `->`, `T *` để sửa, `const T *` để đọc; struct chứa con trỏ: *shallow copy* và *deep copy*; danh sách liên kết.
3. **Truyền hành vi bằng con trỏ hàm:** callback, bảng dispatch, `qsort` với hàm so sánh đúng hợp đồng (không trừ, có khoá phụ).
4. **Cài đặt và phân tích sắp xếp $O(n^2)$:** bubble, selection, insertion; đếm phép so sánh, đo thời gian, nhận biết tính *ổn định*.

Sản phẩm cần nộp

7 chương trình `bai01.c` ... `bai07.c`, biên dịch `-Wall -Wextra -std=c11` **sạch cảnh báo**; Bài 2, 4, 5, 7 (có `malloc`) chạy `valgrind` **sạch lỗi, không rò rỉ**; Bài 6 kèm **nhận xét** về `sizeof` trong báo cáo.

Trả lời câu hỏi của tuần trước: vì sao `struct Node` là 16 byte?

```c
#include <stddef.h>          /* offsetof */

struct Node {
    int data;                /* 4 byte */
    struct Node *next;       /* 8 byte */
};
/* trong main: */
printf("%zu %zu %zu\n", sizeof(struct Node),
       offsetof(struct Node, data),
       offsetof(struct Node, next));
```

```
$ ./node
16 0 8
```



- Con trỏ 8 byte phải nằm ở địa chỉ **chia hết cho 8** $\Rightarrow$ `next` không thể bắt đầu ở byte 4.
- Trình biên dịch chèn **4 byte đệm** (padding) sau `data`: $4 + 4 + 8 = 16$.
- Valgrind ở Lab 3 báo "16 bytes in 1 blocks" cho mỗi nút – chính là con số này.

Thông điệp chính

`sizeof` của struct *không* phải tổng kích thước các trường – không tự cộng, luôn dùng `sizeof`.

Vì sao kiểu dữ liệu nâng cao là chuyện của lập trình hệ thống?

Học ở Lab 4	**Gặp lại ở các tuần sau**
`struct` + con trỏ tới struct	`stat(path, &st)`: `struct stat` (Lab 6)
Bố cục byte, `sizeof`	Ghi/đọc bản ghi bằng `write`/`read`, `lseek` tới bản ghi k (Lab 5)
Con trỏ hàm, callback	`signal(SIGINT, handler)`; `pthread_create(..., fn, arg)` (Lab 7, 9)
`union` có nhãn	Thông điệp IPC nhiều loại (Lab 8)
`enum` trạng thái	Trạng thái tiến trình, mã lệnh của shell (Lab 7)
Danh sách liên kết	Danh sách job của tiny shell (Lab 7)

Bốn công cụ, một ý

struct gom byte thành bản ghi; **union** cho nhiều cách đọc cùng byte; **enum** đặt tên cho con số; **con trỏ hàm** truyền *hành vi* như truyền dữ liệu.

Đã gặp ở Lab 3

`struct Node`, `p->x`, `malloc(sizeof *p)` và `void *` – hôm nay ta học chúng *đến nơi*.

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Vấn đề: dữ liệu của một sinh viên bị tách rời

Mảng song song (`parallel.c`)

```c
int   ids[100];
char  names[100][50];
float gpas[100];

/* doi cho SV 5 va 6 */
float t = gpas[5];
gpas[5] = gpas[6];
gpas[6] = t;
/* QUEN doi ids va names:
   du lieu sai lech, khong
   canh bao nao */
```

Một bản ghi (`student.c`)

```c
struct Student {
    int   id;
    char  name[50];
    float gpa;
};
struct Student roster[100];

/* doi cho: MOT phep gan, ca ban ghi */
struct Student t = roster[5];
roster[5] = roster[6];
roster[6] = t;
```

Ba vấn đề của mảng song song (tài liệu Lab, mục 1.1)

Liên kết mong manh – quan hệ chỉ nằm ở chỉ số *i*; **khó bảo trì** – sắp xếp/xoá phải sửa *mọi* mảng; **khó truyền vào hàm** – mỗi thực thể thành nhiều tham số. `struct` biểu diễn thực thể thành *một giá trị*.

Định nghĩa, khai báo và khởi tạo

```c
struct Student {            /* dinh nghia KIEU */
    char  name[50];
    int   age;
    float gpa;
    int   id;
};                          /* nho dau ; */

struct Student s1 = {"Alice", 20, 3.8f, 1001};
/* C99: khoi tao theo TEN truong */
struct Student s2 = { .id = 1002, .name = "Bob" };
struct Student s3 = {0};    /* moi truong = 0 */
struct Student s4;          /* cuc bo: RAC */

printf("%s %.2f\n", s1.name, s1.gpa);
s1.age = 21;                /* toan tu cham */
```

- Định nghĩa struct tạo **kiểu mới**, chưa cấp bộ nhớ; khai báo biến mới cấp.
- Khởi tạo theo vị trí: đúng *thứ tự khai báo* trường.
- Khởi tạo theo tên (`.id = ...`): không phụ thuộc thứ tự, dễ đọc.
- Có *ít nhất một* bộ khởi tạo $\Rightarrow$ các trường còn lại bằng 0 (`s2.age == 0`).
- Không khởi tạo (`s4`) $\Rightarrow$ biến cục bộ mang **giá trị rác** – như biến `int` ở Lab 2.

Gán struct là *sao chép*; so sánh thì không có ==

```c
struct Student a = {"Alice", 20, 3.8f, 1001};
struct Student b = a;       /* chep MOI byte  */
b.name[0] = 'E';            /* sua ban sao    */
printf("%s %s\n", a.name, b.name);
/* Alice Elice -- a khong doi */

char x[50] = "Hi", y[50];
/* y = x;   LOI: mang khong gan duoc,
   nhung mang NAM TRONG struct thi duoc */

if (a == b) { }             /* LOI bien dich */
```

```
error: invalid operands to binary == (have 'struct Student'
       and 'struct Student')
```

- `b = a` chép **toàn bộ** các trường – kể cả mảng `name[50]`; `b` là bản độc lập.
- Truyền struct vào hàm cũng là *sao chép* (tham trị – Lab 3).
- C **không có** == cho struct: so sánh từng trường, hoặc viết hàm `point_equal`.
- Không dùng `memcmp`: các byte *đệm* mang giá trị rác (mục sau).

Đính chính SP04, trang 13

Chú thích "`backup = s1; // deep copy`" gây hiểu nhầm: đây là sao chép *từng byte*. Nếu struct chứa *con trỏ*, phép gán chỉ chép địa chỉ – **shallow copy** (Bài 5).

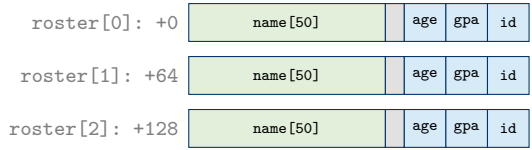
Struct lồng nhau và mảng struct

```c
typedef struct { double x, y; } Point;
typedef struct {
    Point  center;       /* struct long nhau */
    double radius;
} Circle;

Circle c = { {3.0, 4.0}, 5.0 };
printf("cx = %.1f\n", c.center.x);

struct Student roster[100];   /* mang struct */
roster[0].gpa = 3.8f;
struct Student *p = &roster[1];
printf("%td\n", (char *)p - (char *)roster);
```

```
cx = 3.0
64
```



- Truy cập theo chuỗi: `c.center.x`.
- Phần tử mảng struct nằm **liền nhau**; bước nhảy đúng bằng `sizeof(struct Student)` $= 64$, *không phải* $50 + 4 + 4 + 4 = 62$.
- 2 byte xám: đệm để age nằm ở địa chỉ chia hết cho 4.

typedef: đặt tên ngắn cho kiểu

```c
typedef unsigned long long uint64; /* Lab */
#include <stdint.h>          /* nen dung   */
uint64_t big;  int32_t n;    /* co dinh    */

typedef struct {             /* VO DANH    */
    char name[50];
    int  age;
} Student;
Student s1;           /* khong can "struct" */

/* Tu tham chieu: BAT BUOC co ten the */
typedef struct Node {
    int data;
    struct Node *next;  /* chua co "Node" */
} Node;
```

- typedef *không* tạo kiểu mới – chỉ là **bí danh**.
- Kiểu có độ rộng cố định nên lấy từ <stdint.h>: uint64_t thống nhất trên mọi máy (nhớ: long là 8 byte trên Ubuntu nhưng 4 byte trên Windows).
- Struct vô danh không tự tham chiếu được:

```
typedef struct { int data; Node *next; } Node;
error: unknown type name 'Node'
```

Bài 7 dùng đúng struct Node có tên thẻ như đề – và typedef là tuỳ chọn.

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Ba quy tắc căn lề (alignment) trên x86-64

1. Mỗi trường bắt đầu ở **offset chia hết cho căn lề của nó** (`_Alignof`); thiếu thì chèn *đệm* trước trường.
2. Căn lề của struct = căn lề **lớn nhất** trong các trường.
3. `sizeof` của struct được làm tròn lên **bội của căn lề struct** (đệm cuối) – để mọi phần tử của *mảng* struct đều thẳng hàng.

Căn lề ≠ kích thước

SP04 (trang 14) viết "multiple of its *size*". Đúng với kiểu cơ bản trên x86-64, sai với mảng và struct: `char name[50]` căn lề 1, `Point` (2 `double`, 16 byte) căn lề 8.

Kiểu	`sizeof`	`_Alignof`
`char`	1	1
`int`, `float`	4	4
`double`	8	8
`void *`	8	8
`char[50]`	50	1
`struct Node`	16	8

Đo bằng `layout.c` (gcc 13, x86-64). Ubuntu x86-64 cho cùng kết quả với các kiểu này (khác duy nhất: `long` 8 byte trên Linux, 4 byte trên Windows).

Thí nghiệm `layout.c`: đệm nằm ở đâu?

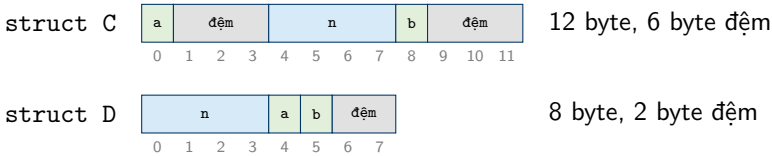
```c
#include <stddef.h>          /* offsetof */
struct A { int x; int y; };
struct B { char c; int n; };
struct C { char a; int n; char b; };
struct D { int n; char a; char b; };
struct E { char c; double d; };

#define SHOW(T) printf("%-9s %2zu %zu\n", \
        #T, sizeof(T), _Alignof(T))
/* trong main: */
SHOW(struct A); SHOW(struct B); SHOW(struct C);
SHOW(struct D); SHOW(struct E);
printf("C: a@%zu n@%zu b@%zu\n",
       offsetof(struct C, a),
       offsetof(struct C, n),
       offsetof(struct C, b));
```

```
$ gcc -Wall -std=c11 layout.c -o layout
$ ./layout
struct A   8 4
struct B   8 4
struct C  12 4
struct D   8 4
struct E  16 8
C: a@0 n@4 b@8
```

- `offsetof(T, f)` (`<stddef.h>`): vị trí byte của trường `f` – công cụ để *kiểm chứng*, không đoán.
- `_Alignof` là từ khoá C11 (`alignof` trong `<stdalign.h>`).
- `struct E`: `double` cần offset 8 $\Rightarrow$ 7 byte đệm, chỉ để chứa 1 `char`.

Cùng ba trường, hai thứ tự: 12 byte hay 8 byte



Quy tắc thực hành

Khai báo trường theo căn lề **giảm dần** (con trỏ, `double` → `int` → `short` → `char`). Mảng 1 triệu `struct C`: 12 MB; `struct D`: 8 MB.

Đính chính SP04, trang 14

Chú thích mã ghi `sizeof(C) = 8` (1+1+2pad+4; đúng là **12** – như chính hình vẽ bên cạnh trong slide.

Đệm là byte rác – và vì sao CPU cần căn lề

```
(gdb) print b
$1 = {c = 65 'A', n = 7}
(gdb) print sizeof(b)
$2 = 8
(gdb) x/8xb &b
0x7fffffffe0c8: 0x41 0x18 0xd6 0xd3 0x07 0x00 0x00 0x00
```

Minh hoạ: địa chỉ và 3 byte đệm (0x18 0xd6 0xd3) khác trên máy bạn.

- Byte 1–3 là **rác**: `b.c = 'A'` chỉ ghi byte 0.
- $\Rightarrow$ `memcmp(&a, &b, sizeof a)` có thể báo khác dù mọi trường bằng nhau.
- 07 00 00 00: `n = 7` lưu *byte thấp trước* (little-endian).

Vì sao không xếp khít?

CPU đọc bộ nhớ theo khối thẳng hàng. Một `int` vắt qua ranh giới khối cần *hai* lần đọc; trên một số kiến trúc (ARM cũ, SPARC) truy cập lệch còn gây lỗi *bus error*. Đệm đổi một ít bộ nhớ lấy tốc độ và tính đúng.

Nối sang Lab 5

Ghi cả mảng struct ra file bằng `write` sẽ ghi *cả đệm*; file phụ thuộc trình biên dịch và máy. `__attribute__((packed))` bỏ đệm nhưng chậm và nguy hiểm – chỉ dùng cho định dạng file/gói tin cố định.

Nội dung

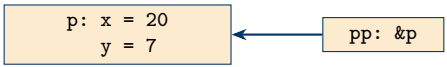
1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Toán tử . và ->

```c
typedef struct { int x; int y; } Point;

Point  p  = {3, 7};
Point *pp = &p;

p.x    = 10;     /* bien struct: dau cham */
pp->x  = 20;     /* con tro: mui ten      */
(*pp).x = 20;    /* tuong duong pp->x     */
*pp.x  = 20;     /* SAI: = *(pp.x)        */
```



- . khi bên trái là **biến struct**; -> khi bên trái là **con trỏ tới struct**.
- . ưu tiên cao hơn *: *pp.x được hiểu là *(pp.x) – lý do C có ->.
- Dùng nhầm, gcc báo lỗi và còn gợi ý:

```
error: invalid type argument of '->' (have 'Point')
error: 'pp' is a pointer; did you mean to use '->'?
```

`n->data`, `n->next` trong `list.c` (Lab 3) chính là toán tử này.

Truyền struct theo giá trị hay theo con trỏ (scale.c)

```c
void scaleBAD(Point p, int factor) { /* SAO */
    p.x *= factor;
    p.y *= factor;
}
void scale(Point *p, int factor) {   /* GOC */
    p->x *= factor;
    p->y *= factor;
}
Point scaled(Point p, int factor) {  /* MOI */
    p.x *= factor;
    p.y *= factor;
    return p;
}

Point p = {3, 7};
scaleBAD(p, 2);          /* p = (3, 7)    */
scale(&p, 2);            /* p = (6, 14)   */
Point q = scaled(p, 10); /* q = (60, 140) */
                         /* p van (6, 14) */
```

- Đúng bài swap của Lab 3, giờ với struct: muốn sửa bản gốc ⇒ truyền **địa chỉ**.
- `scaled` trả về struct: hợp lệ và an toàn (trả *giá trị*, không trả địa chỉ biến cục bộ).

Đính chính SP04, trang 20

`printf("(%d, %d)", p->x, p->y)` với `p` là `Point` **không biên dịch được** (cần `p.x`). `scalePtr(&p, 3)` chỉ nhân `x`: kết quả `p.x = 9, p.y = 7` (slide ghi `p.x=6`).

Chi phí sao chép và const T *

```c
/* 3 double = 24 byte */
typedef struct { double x, y, z; } Vec3;

double len2(Vec3 v);          /* chep 24 B */
double len2p(const Vec3 *v);  /* chep 8 B  */

void printPoint(const Point *p) {
    printf("(%d, %d)\n", p->x, p->y);
    /* p->x = 0;  LOI bien dich:
       assignment of member 'x'
       in read-only object */
}
```

Quy tắc: struct nhỏ, chỉ đọc → truyền giá trị cũng được; struct lớn hoặc cần sửa → `T *`; chỉ đọc → `const T *`.

Khai báo	Sửa *p?	Đổi p?
`Point *p`	được	được
`const Point *p`	**không**	được
`Point *const p`	được	**không**
`const Point *const p`	không	không

Đọc từ phải sang trái (Lab 3): `const Point *p` là "con trỏ tới `Point` *hằng*".

Thuật ngữ (tài liệu Lab, mục 2)

"const pointer to struct" thực ra là `Point *const p`. Tham số chỉ đọc ta dùng là **con trỏ tới hằng** `const Point *`.

Struct trên heap và ba lỗi quen thuộc

```c
Point *p = malloc(sizeof *p);   /* 1 Point */
if (p == NULL) {
    perror("malloc");
    return 1;
}
p->x = 10;
p->y = 20;
/* ... */
free(p);                    /* tai lieu thieu */
p = NULL;

Student *list =
    malloc((size_t)n * sizeof *list);
list[i].score = 9.0;  /* phan tu: dau cham */
```

1. **Quên kiểm tra `NULL`**: `p->x` với `p == NULL` ⇒ `Segmentation fault`.
2. **Trả địa chỉ struct cục bộ**: `Point *make(void) { Point q; return &q; }` – lỗi của Lab 3, nay với struct. Trả *giá trị* `Point`, hoặc `malloc`.
3. **`sizeof` nhầm**: `malloc(sizeof(p))` chỉ cấp 8 byte (kích thước *con trỏ*). Mẫu `sizeof *p` không bao giờ sai.

Nhắc lại

`list[i]` là một `Student` ⇒ `list[i].score`; `&list[i]` hay `list + i` là `Student *` ⇒ `(list + i)->score`.

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

enum: đặt tên cho các hằng số nguyên

```c
typedef enum {
    STATUS_NORMAL,      /* 0 */
    STATUS_WARNING,     /* 1 */
    STATUS_ERROR,       /* 2 */
    STATUS_COUNT        /* 3 = so trang thai */
} SensorStatus;

/* ten in ra: cung thu tu voi enum */
static const char *const
    STATUS_NAMES[STATUS_COUNT] =
    { "NORMAL", "WARNING", "ERROR" };

/* BLUE = 6: dem tiep tu GREEN */
typedef enum { RED, GREEN = 5, BLUE } Color;

int count[STATUS_COUNT] = {0};
count[s]++;                 /* enum lam CHI SO */
printf("%s\n", STATUS_NAMES[s]);
```

- Mặc định đánh số từ 0, tăng 1; gán tường minh thì đếm tiếp từ giá trị đó.
- Phần tử cuối `STATUS_COUNT` là mẹo phổ biến: tự bằng số trạng thái – dùng làm kích thước mảng (Bài 4).
- Mảng tên song song với enum để in ra chữ: `printf` chỉ thấy số nguyên.
- Hằng enum nằm trong **phạm vi toàn cục**: đặt tiền tố (`STATUS_`) để tránh trùng – ví dụ `ERROR` đã là macro trong `<windows.h>`.

gcc kiểm tra gì với enum? Đo thật (enumsw.c)

```c
typedef enum {
    SUN, MON, TUE, WED, THU, FRI, SAT
} Day;

int is_weekend(Day d) {
    switch (d) {              /* quen FRI */
    case SAT: case SUN:
        return 1;
    case MON: case TUE: case WED: case THU:
        return 0;
    }
    return 0;
}

Day bad = 42;                 /* khong canh bao */
```

```
$ gcc -Wall -Wextra -std=c11 enumsw.c
enumsw.c:11:5: warning: enumeration value 'FRI' not handled
in switch [-Wswitch]
```

- `-Wswitch` (có trong `-Wall`) báo `switch` trên enum **thiếu nhãn** – lợi ích thật so với `#define`.
- Nhưng `Day bad = 42;` **không** có cảnh báo nào: trong C, hằng enum có kiểu `int`, biến enum chỉ là số nguyên.
- Viết `switch` không có `default` để gcc nhắc khi thêm trạng thái mới.

Tài liệu Lab, mục 1.3

"enum ... giúp trình biên dịch có thể cảnh báo khi sử dụng sai giá trị" – chỉ đúng với `switch`; gán số ngoài miền thì C không kiểm tra. Dữ liệu từ bên ngoài phải tự kiểm tra (Bài 4).

union: nhiều trường, *một* vùng nhớ

```c
union Data {                    /* tai lieu Lab */
    int   i;
    float f;
    char  str[20];
};
union Odd {
    char s[5];
    int  i;
};
printf("%zu %zu\n", sizeof(union Data),
                    sizeof(union Odd));
```

```
20 8
```



- Mọi trường bắt đầu ở **cùng địa chỉ** (offset 0); ghi trường này đè lên trường kia.
- `sizeof(union)` $\geq$ trường lớn nhất, **làm tròn lên bội căn lề**: `union Odd` có trường lớn nhất 5 byte nhưng `sizeof` $= 8$.
- Struct: *tổng* (cộng đệm). Union: *lớn nhất* (cộng đệm).

Tài liệu Lab, mục 1.4; SP04, trang 16

"Kích thước của union **bằng** kích thước của trường lớn nhất" – chỉ đúng khi trường lớn nhất đã là bội của căn lề (như `union Data`: 20).

Đọc cùng byte theo kiểu khác (unionb.c – sửa ví dụ SP04)

```c
union Number {
    int i;
    float f;
    unsigned char bytes[4];
};
union Number n;
n.i = 0x00004100;
printf("n.i = %d\n", n.i);
printf("bytes: %02X %02X %02X %02X\n",
       n.bytes[0], n.bytes[1],
       n.bytes[2], n.bytes[3]);
printf("(char)n.i = %d, bytes[1] = '%c'\n",
       (char)n.i, n.bytes[1]);
printf("n.f = %g\n", n.f);
n.f = 1.0f;
printf("n.i = 0x%08X\n", (unsigned)n.i);
```

```
n.i = 16640
bytes: 00 41 00 00
(char)n.i = 0, bytes[1] = 'A'
n.f = 2.33176e-41
n.i = 0x3F800000
```

- Little-endian: byte thấp 0x00 đứng đầu; 0x41 = 'A' ở bytes[1].
- SP04 in printf("%c", n.i), chú thích "'A'": %c lấy byte thấp 0x00 – in ký tự rỗng.
- Đọc trường khác trường vừa ghi: trong C là *diễn giải lại byte* (type punning, C11 §6.5.2.3), không phải "undefined behavior" như SP04 ghi (C++ mới cấm).

Union có nhãn (tagged union): luôn biết trường nào hợp lệ

```c
typedef enum {
    TYPE_INT, TYPE_FLOAT, TYPE_STR
} VType;
typedef struct {
    VType tag;                  /* NHAN */
    union { int i; float f; char *s; } val;
} Value;

void print_value(const Value *v) {
    switch (v->tag) {        /* doc NHAN truoc */
    case TYPE_INT:
        printf("%d\n", v->val.i); break;
    case TYPE_FLOAT:
        printf("%g\n", v->val.f); break;
    case TYPE_STR:
        printf("%s\n", v->val.s); break;
    }
}
Value v = { .tag = TYPE_INT, .val.i = 42 };
print_value(&v);                /* 42 */
```



`sizeof(Value)` = 16 (đo thật)

- Ghi trường nào thì đặt `tag` tương ứng; đọc thì `switch` trên `tag`.
- Đây là cách JSON parser, trình biên dịch (token), thông điệp IPC biểu diễn "một trong nhiều loại".

SP04, trang 18

Ví dụ ghi `v.val.i` rồi in `v.val.f` dưới tiêu đề "Why is this safe?" – đó là cách dùng *sai*; an toàn chỉ có khi `tag` được kiểm tra như bên trái.

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Hàm cũng có địa chỉ: con trỏ hàm

```c
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int (*op)(int, int);    /* con tro toi ham   */
int *f(int, int);       /* HAM tra ve int *  */

op = add;               /* = &add            */
printf("%d\n", op(3, 4));      /* 7  */
op = mul;
printf("%d\n", (*op)(3, 4));   /* 12 */
```

`op(3, 4)` và `(*op)(3, 4)` như nhau; `add` và `&add` như nhau.

Đọc khai báo `int (*op)(int, int)` từ tên ra ngoài:

1. `op` là ...
2. `(*op)` ... một **con trỏ** tới ...
3. `(int, int)` ... hàm nhận hai `int` ...
4. `int` ... và trả về `int`.

Cặp ngoặc quyết định tất cả

Bỏ ngoặc: `int *op(int, int)` là *khai báo hàm* trả về `int *` – quy tắc đọc khai báo con trỏ của Lab 3.

typedef, callback và bảng dispatch (fnptr.c)

```c
typedef int (*BinOp)(int, int);  /* ten kieu */

int apply(int a, int b, BinOp fn) { /* callback */
    return fn(a, b);
}
printf("%d\n", apply(5, 3, sub));    /* 2 */

/* Bang dispatch: chon ham theo chi so */
const BinOp table[] = { add, sub, mul };
const char *names[] = { "add", "sub", "mul" };
for (int k = 0; k < 3; k++)
    printf("%s(6, 2) = %d\n",
           names[k], table[k](6, 2));
```

```
add(6, 2) = 8
sub(6, 2) = 4
mul(6, 2) = 12
```

- apply không biết phép toán là gì – **tách thuật toán khỏi hành vi**.
- **Callback**: hàm truyền vào để được *gọi lại*: hàm so sánh của qsort, signal(SIGINT, handler), hàm luồng của pthread_create.
- **Bảng dispatch** thay chuỗi if/switch dài: menu Bài 2 có thể viết bằng một mảng con trỏ hàm.

Hai bẫy của con trỏ hàm

1. Chưa khởi tạo (tài liệu Lab, mục 3)

```
BinOp fn;           /* cuc bo: RAC, khong NULL */
if (fn != NULL)     /* kiem tra VO ICH         */
    fn(1, 2);       /* nhay toi dia chi rac    */

BinOp g = NULL;     /* DUNG: khoi tao truoc    */
if (g != NULL)
    g(1, 2);
```

Tài liệu khai báo `BinOp fn;` rồi khuyên kiểm tra `fn != NULL`: phép kiểm tra chỉ có nghĩa khi `fn` đã được *gán* `NULL`. gcc 13 `-Wall` báo: `'fn' is used uninitialized`.

2. Sai chữ ký

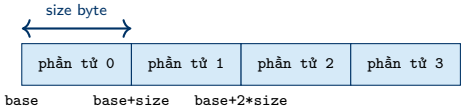
```
int cmp_int(int *a, int *b) {   /* SAI kieu */
    return (*a > *b) - (*a < *b);
}
qsort(v, 3, sizeof v[0], cmp_int);
```

```
warning: passing argument 4 of 'qsort' from incompatible
pointer type [-Wincompatible-pointer-types]
note: expected 'int (*)(const void *, const void *)' but
argument is of type 'int (*)(int *, int *)'
```

Gọi hàm qua con trỏ sai kiểu là hành vi không xác định. Từ gcc 14 đây là **lỗi**, không còn là cảnh báo.

qsort: một thuật toán, mọi kiểu dữ liệu

```c
#include <stdlib.h>
void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));
```



- `qsort` **không biết kiểu**: chỉ biết địa chỉ đầu, số phần tử, số byte mỗi phần tử.
- Nó tự hoán đổi các khối `size` byte; muốn biết "ai đứng trước" thì **gọi lại** `compar`.

Hợp đồng của hàm so sánh

Nhận *địa chỉ hai phần tử*; trả **âm** nếu phần tử 1 đứng trước, **0** nếu ngang nhau, **dương** nếu đứng sau. Kết quả phải **nhất quán** (thứ tự toàn phần, bắc cầu) – vi phạm là hành vi không xác định.

`void *` + con trỏ hàm = lập trình tổng quát kiểu C. Cùng mẫu: `bsearch`, `pthread_create`.

Viết hàm so sánh đúng mẫu

```c
int cmp_int(const void *pa, const void *pb) {
    int x = *(const int *)pa;
    int y = *(const int *)pb;
    return (x > y) - (x < y);          /* -1, 0, 1 */
}

int cmp_gpa_desc(const void *pa, const void *pb) {
    const Student *a = pa, *b = pb;    /* tu doi kieu */
    if (a->gpa < b->gpa) return 1;     /* GIAM dan    */
    if (a->gpa > b->gpa) return -1;
    return 0;
}

int cmp_name(const void *pa, const void *pb) {
    const Student *a = pa, *b = pb;
    return strcmp(a->name, b->name);
}

qsort(roster, n, sizeof roster[0], cmp_gpa_desc);
```

- Ép về `const T *` – giữ `const`, đừng ép về `T *` như ví dụ trong tài liệu.
- `(x > y) - (x < y)`: ngắn, không tràn.
- **Số thực**: luôn so sánh <, > tường minh. `return a->gpa - b->gpa;` đổi hiệu 0.3 thành `int` 0 – "bằng nhau".
- **Giảm dần**: đảo vai trò hai đối số, không nhân -1.
- `size`: viết `sizeof roster[0]`, không `sizeof roster` hay `sizeof(Student *)`.

Vì sao không viết `return a - b`? Đo thật (`cmpbad.c`)

```c
/* SAI: co the tran so */
int cmp_sub(const void *a, const void *b) {
    return *(const int *)a - *(const int *)b;
}

int x = INT_MIN, y = 1;
printf("%d\n", cmp_sub(&x, &y));

int v[] = { 5, INT_MIN, 3, INT_MAX, -2 };
qsort(v, 5, sizeof v[0], cmp_sub);
```

```
cmp_sub(INT_MIN, 1) = 2147483647
cmp_sub: 3 5 2147483647 -2147483648 -2
cmp_ok : -2147483648 -2 3 5 2147483647
```

- `INT_MIN` − 1 tràn `int` (hành vi không xác định); trên thực tế quay vòng thành số *dương* lớn nhất ⇒ "`INT_MIN` > 1".
- Hàm so sánh không còn bắc cầu ⇒ `qsort` cho thứ tự sai, *không báo lỗi*.
- Dòng `cmp_sub`: cụ thể phụ thuộc thư viện (đo trên máy soạn); dòng `cmp_ok` luôn đúng.
- Với dữ liệu nhỏ (điểm 0–10) phép trừ "chạy được" – vì thế lỗi này sống lâu. Dùng mẫu đúng từ đầu.

qsort *không* hứa giữ thứ tự phần tử bằng nhau (stable.c)

```c
/* da xep theo ten: An < Binh < Cuong < ... */
Rec r[] = { {"An", 8},   {"Binh", 7}, {"Cuong", 8},
            {"Dung", 7}, {"Em", 8},   {"Giang", 7} };
qsort(r, 6, sizeof r[0], by_score);

/* Sua: them KHOA PHU (thu tu nhap) */
int by_score_then_order(const void *a, const void *b) {
    const Rec2 *x = a, *y = b;
    if (x->score != y->score)
        return (x->score > y->score)
             - (x->score < y->score);
    return (x->order > y->order)
         - (x->order < y->order);
}
```

```
qsort theo diem  : Binh(7) Dung(7) Giang(7) Em(8) Cuong(8) An(8)
qsort + khoa phu : Binh(7) Dung(7) Giang(7) An(8) Cuong(8) Em(8)
```

- Đo trên máy soạn (thư viện C của Windows): ba bạn 8 điểm bị **đảo ngược** thứ tự tên.
- Chuẩn C và `man 3 qsort`: thứ tự các phần tử bằng nhau là *không xác định*. Trên Ubuntu có thể bạn thấy thứ tự được giữ – đó là chi tiết cài đặt, **không phải bảo đảm**.
- Cần thứ tự xác định $\Rightarrow$ thêm **khoá phụ**, hoặc dùng thuật toán *ổn định* (insertion sort – Bài 2).

Ba bẫy khác với qsort và bsearch

```c
/* 1. Mang con tro: phan tu la char *,
      comparator nhan char ** */
const char *w[] = { "pear", "apple", "fig" };
int cmp_str(const void *a, const void *b) {
    const char *const *x = a, *const *y = b;
    return strcmp(*x, *y);    /* KHONG strcmp(a, b) */
}
qsort(w, 3, sizeof w[0], cmp_str);

/* 3. bsearch: mang DA xep, CUNG comparator */
int key = 8;
int *hit = bsearch(&key, v, n, sizeof v[0], cmp_int);
if (hit != NULL)
    printf("o vi tri %td\n", hit - v);
```

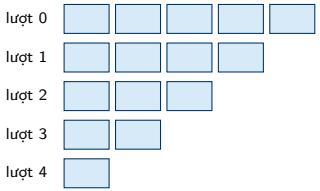
1. `compar` nhận **địa chỉ phần tử**. Phần tử là `char *` thì đối số là `char **`: `strcmp(a, b)` so sánh *các con trỏ*, cho kết quả rác.
2. `size` sai (`sizeof(Student *)` $= 8$ thay cho `sizeof(Student)`): `qsort` cắt bản ghi thành mảnh – dữ liệu bị trộn.
3. `bsearch`: tìm nhị phân $O(\log n)$ của Lab 3, bản tổng quát. Chỉ đúng trên mảng *đã sắp xếp* bằng *cùng* hàm so sánh.

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Đếm phép toán: vì sao $O(n^2)$

- Đếm hai đại lượng: **phép so sánh** hai phần tử và **lần ghi** phần tử vào mảng.
- Hai vòng lồng nhau, vòng trong ngắn dần: $(n-1)+\cdots+1 = n(n-1)/2$ phép so sánh; $n = 1000$: 499 500.
- $O(n^2)$: tăng n lên 10 lần $\Rightarrow$ thời gian tăng ≈ 100 lần.



mỗi ô = 1 phép so sánh
$n = 6$: $5+4+3+2+1 = 15$

Tiêu chí khi chọn thuật toán

Số **so sánh**, số **lần ghi**, **ổn định** (giữ thứ tự phần tử bằng nhau), **thích nghi** (nhanh hơn khi dữ liệu gần đúng thứ tự).

Bubble sort: nổi bọt từng cặp kề nhau

```c
void bubble_sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
                swapped = 1;
            }
        if (!swapped) break;   /* dung som */
    }
}
/* Bat bien: sau luot i, a[n-1-i .. n-1]
   da dung vi tri cuoi cung */
```

Lượt 1 với {5, 2, 8, 1, 9} (tài liệu Lab, mục 5.1):

So sánh	Mảng sau bước
$5 > 2$ đổi	2 5 8 1 9
5, 8 giữ	2 5 8 1 9
$8 > 1$ đổi	2 5 1 8 9
8, 9 giữ	2 5 1 8 9

- Xấu nhất $O(n^2)$; có cờ `swapped`: tốt nhất $O(n)$ (mảng đã sắp).
- **Ổn định**: chỉ đổi khi > (không đổi khi bằng).
- Nhiều phép ghi nhất – chủ yếu dùng để dạy.

Selection sort: chọn nhỏ nhất, đặt vào đầu

```c
void selection_sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min])
                min = j;
        if (min != i) {
            int t = a[i];
            a[i] = a[min];
            a[min] = t;
        }
    }
}
/* Bat bien: a[0..i-1] nho nhat, da dung cho */
```

Vì sao không ổn định? Ví dụ của SP04: [5_a, 3, 8, 5_b, 1]

Lượt	Mảng
1: đổi $5_a \leftrightarrow 1$	1 3 8 5_b 5_a
2: 3 đã đúng	1 3 8 5_b 5_a
3: đổi $8 \leftrightarrow 5_b$	1 3 5_b 8 5_a
4: đổi $8 \leftrightarrow 5_a$	1 3 5_b 5_a 8

- Phép đổi "nhảy xa" đưa 5_a ra sau 5_b.
- Luôn $n(n-1)/2$ so sánh – **không thích nghi**.
- Tối đa $n-1$ lần đổi: hợp khi *ghi* đắt (flash).

Insertion sort: chèn như xếp bài trên tay

```c
void insertion_sort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];      /* bai can chen */
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];   /* dich phai */
            j--;
        }
        a[j + 1] = key;      /* chen vao lo  */
    }
}
/* Bat bien: a[0..i-1] da sap xep
   (nhung CHUA o vi tri cuoi cung) */
```

`{5, 2, 8, 1, 9}`:

key	Mảng sau khi chèn
2	2 5 | 8 1 9
8	2 5 8 | 1 9
1	1 2 5 8 | 9
9	1 2 5 8 9 (tài liệu bỏ sót bước này)

- **Ổn định**: dừng khi `a[j] <= key`, phần tử bằng nhau không vượt qua nhau.
- **Thích nghi**: số lần dịch = số *cặp nghịch thế*; gần sắp xếp $\Rightarrow$ gần $O(n)$.
- Dùng bên trong Timsort, introsort cho đoạn nhỏ.

Đo thật: số phép so sánh và số lần ghi ($n = 1000$, `sorts.c`)

	Ngẫu nhiên		Đã sắp xếp		Ngược	
	so sánh	ghi	so sánh	ghi	so sánh	ghi
bubble	499 364	500 096	**999**	0	499 500	999 000
selection	499 500	1 990	499 500	0	499 500	1 000
insertion	251 039	251 047	**999**	999	499 500	500 499
`qsort`	11 097	–	9 684	–	9 684	–

Đếm trong mã (`Stats *st` – tham số đầu ra là một struct); `qsort`: đếm số lần gọi hàm so sánh. Dữ liệu sinh bằng xorshift với hạt giống cố định: chạy lại trên Ubuntu cho *cùng* các số đếm của 3 thuật toán tự viết; hàng `qsort` phụ thuộc thư viện (ở đây: thư viện C của Windows).

- Ngẫu nhiên: bubble và insertion cùng dịch $\approx 250\,048$ *cặp nghịch thế* $\approx n^2/4$; insertion chỉ so sánh một nửa.
- Selection: luôn 499 500 so sánh nhưng chỉ ≤ 999 lần đổi.
- Đã sắp xếp: bubble (có cờ) và insertion chỉ $n - 1$ so sánh.
- `qsort`: $\approx n \log_2 n \approx 10^4$.

Đo thật: thời gian chạy – và đính chính SP04

n	**bubble**	**selection**	**insertion**	`qsort`
1 000	1,7 ms	0,9 ms	0,12 ms	0,06 ms
10 000	162 ms	94 ms	12 ms	0,74 ms
50 000	5 431 ms	2 370 ms	311 ms	4,2 ms

Dữ liệu ngẫu nhiên, `gcc -O2`, đo bằng `timespec_get` trên máy soạn (Ryzen 5 5600H). Số tuyệt đối trên máy lab sẽ khác, *tỉ lệ* thì không.

n tăng 5 lần ($10\,000 \to 50\,000$): selection, insertion chậm ≈ 25 lần – đúng $O(n^2)$.

SP04, trang 9

Bảng ghi $n = 10\,000$: 10^8 phép toán, "< 1 ms" với 10^9 phép/giây. Đúng là $10^8/10^9 = 0{,}1$ giây $= 100$ ms – khớp với 94–162 ms đo được.

Kết luận

$O(n^2)$ chấp nhận được tới vài nghìn phần tử. Dữ liệu thật: dùng `qsort`. Đoạn nhỏ, gần đúng thứ tự, cần ổn định: insertion sort.

Sắp xếp tổng quát tự viết: thuật toán nhận con trỏ hàm

Mức 1 – kiểu cụ thể (Bài 2)

```
typedef int (*StudentCmp)(const Student *, const Student *);

void sort_students(Student *a, int n, StudentCmp cmp) {
    for (int i = 1; i < n; i++) {
        Student key = a[i];        /* chep ca ban ghi */
        int j = i - 1;
        while (j >= 0 && cmp(&a[j], &key) > 0) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
sort_students(list, n, cmp_name_asc);
sort_students(list, n, cmp_score_desc);
```

Mức 2 – mọi kiểu, như qsort (isort.c)

```
int isort(void *base, size_t n, size_t size,
          int (*cmp)(const void *, const void *)) {
    char *a = base;            /* so hoc theo BYTE */
    char *key = malloc(size);
    if (key == NULL) return -1;
    for (size_t i = 1; i < n; i++) {
        memcpy(key, a + i * size, size);
        size_t j = i;
        while (j > 0 && cmp(a + (j - 1) * size, key) > 0) {
            memcpy(a + j * size, a + (j - 1) * size, size);
            j--;
        }
        memcpy(a + j * size, key, size);
    }
    free(key);
    return 0;
}
```

Mức 2 là cách qsort được viết: không biết kiểu, chỉ chép khối size byte bằng memcpy và hỏi cmp. Vì insertion sort ổn định: xếp theo *tên* rồi theo *điểm* $\Rightarrow$ cùng điểm thì vẫn theo tên.

Nội dung

1 Mở đầu: từ Lab 3 sang Lab 4

2 Struct và typedef

3 Bố cục bộ nhớ: căn lề và đệm

4 Con trỏ tới cấu trúc

5 Enum và union

6 Con trỏ hàm và `qsort`

7 Thuật toán sắp xếp $O(n^2)$

8 Struct chứa con trỏ, danh sách liên kết

9 Lỗi thường gặp và gỡ lỗi

10 Bài tập thực hành

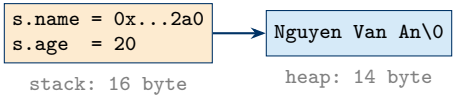
11 Tổng kết

Struct chứa con trỏ: chuỗi nằm *ngoài* struct

```c
struct Student {
    char *name;          /* chi la DIA CHI */
    int   age;
};

char buf[256];
scanf(" %255[^\n]", buf); /* ca dau cach */
size_t len = strlen(buf) + 1;   /* + '\0' */
s.name = malloc(len);
if (s.name == NULL) {
    perror("malloc");
    return 1;
}
memcpy(s.name, buf, len);
s.age = 20;
printf("%zu %zu\n", sizeof s, len);
```

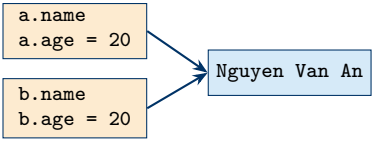
```
16 14            (ho ten: "Nguyen Van An")
```



- `sizeof s` = 16 *không đổi* dù tên dài bao nhiêu: struct chỉ giữ con trỏ.
- Cấp `strlen + 1` byte – quên `+ 1` là ghi tràn 1 byte (Valgrind báo `Invalid write`).
- `strdup` gọn hơn nhưng thuộc POSIX/C23, không có trong `-std=c11` thuần.

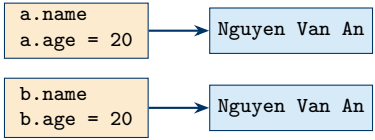
Shallow copy và deep copy (Bài 5)

Shallow: `struct Student b = a;`



Chép *con trỏ*: hai struct cùng trỏ **một** chuỗi. Sửa `b.name[0]` thì `a.name` đổi theo; `free` cả hai là **double free**.

Deep: `student_deep_copy(&b, &a);`



Cấp vùng mới và chép *nội dung*: hai bản độc lập, mỗi bản `free` riêng.

Quy tắc sở hữu (nối Lab 3: "ai cấp phát, ai giải phóng")

Struct có con trỏ cần **bộ ba hàm**: `init` (cấp phát), `copy` (deep copy), `free` (giải phóng). Phép gán `=` và truyền tham trị luôn là shallow – C không tự gọi hàm sao chép.

Double free: Valgrind và glibc nói gì

Sửa Bài 5: thay dòng 68 `shallow.name = NULL;` bằng `student_free(&shallow);`

```
$ valgrind ./bai05 < tests/bai05.in
==5123== Invalid free() / delete / delete[] / realloc()
==5123==    at 0x484988F: free (vg_replace_malloc.c:989)
==5123==    by 0x1094E2: student_free (bai05.c:90)
==5123==    by 0x1093C5: main (bai05.c:68)
==5123==  Address 0x4a9a0a0 is 0 bytes inside a block of size 14 free'd
==5123==    at 0x484988F: free (vg_replace_malloc.c:989)
==5123==    by 0x1094E2: student_free (bai05.c:90)
==5123==    by 0x1093B9: main (bai05.c:67)
==5123==  Block was alloc'd at
==5123==    at 0x4846828: malloc (vg_replace_malloc.c:446)
==5123==    by 0x10945B: student_init (bai05.c:76)
==5123==    by 0x109300: main (bai05.c:40)
```

```
$ ./bai05 < tests/bai05.in       # khong co valgrind
...
free(): double free detected in tcache 2
Aborted (core dumped)
```

Minh hoạ dạng chuẩn Ubuntu x86-64; PID, địa chỉ khác trên máy bạn.

Đọc báo cáo theo ba khối:

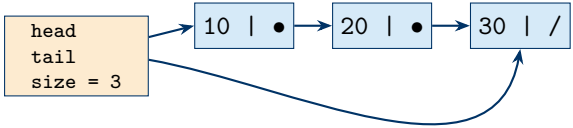
1. **Ở đâu sai**: `free` lần 2, gọi từ dòng 68.
2. **Đã free lúc nào**: dòng 67 – cùng khối 14 byte.
3. **Cấp phát ở đâu**: `student_init`, dòng 76.

Ba khối cùng trỏ *một* vùng $\Rightarrow$ hai struct chia sẻ một chuỗi: dấu hiệu của shallow copy.

Danh sách liên kết lần hai: struct bọc danh sách (Bài 7)

```c
struct Node {
    int data;
    struct Node *next;
};
typedef struct {
    struct Node *head;
    struct Node *tail;      /* nut cuoi */
    int size;
} List;

int list_append(List *list, int value) {
    struct Node *n = create_node(value);
    if (n == NULL) return -1;
    if (list->tail == NULL) list->head = n;
    else                    list->tail->next = n;
    list->tail = n;
    list->size++;
    return 0;
}
```



- Lab 3: `insert_head(&head, v)` – thêm *đầu*, cần `struct Node **`.
- Lab 4: thêm *cuối* giữ đúng thứ tự nhập; giữ `tail` nên $O(1)$ thay vì duyệt $O(n)$.
- Truyền `List *`: một con trỏ thay cho ba biến – lợi ích của struct.
- `list_free` lấy `p->next` *trước* `free(p)` (Lab 3).

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Triệu chứng → nguyên nhân → công cụ

Triệu chứng	**Nguyên nhân hay gặp**	**Công cụ / cách thấy**
`sizeof` lớn hơn tổng các trường	đệm căn lề – *không phải lỗi*	`offsetof`, `gdb x/16xb`
Hàm "sửa" struct nhưng bản gốc không đổi	truyền struct theo giá trị	đổi thành `T *`; `gdb print &p` ở hai frame
`invalid type argument of '->'`	dùng `->` với biến struct (hoặc `.` với con trỏ)	đọc gợi ý của gcc
Sửa bản sao, bản gốc cũng đổi; crash ở `free`	shallow copy struct chứa con trỏ	Valgrind: `Invalid free`, cùng một khối
`qsort` xếp sai, không báo lỗi	comparator trừ (tràn), đọc sai kiểu phần tử, `size` sai	in vài lần gọi comparator; `-fsanitize=undefined`
Thứ tự các bản ghi bằng nhau "nhảy lung tung"	`qsort` không ổn định	thêm khoá phụ
Union in ra số vô nghĩa	đọc trường khác trường vừa ghi	thêm nhãn `enum`, `switch`
`switch` bỏ sót trạng thái	thêm hằng enum mới	`-Wall` (`-Wswitch`)
Tên bị cắt / vòng lặp nhập bị lệch	`scanf("%s")` dừng ở dấu cách; tên dài hơn bộ đệm	`" %49[^\n]"` + kiểm tra ký tự còn lại
`Segmentation fault` khi duyệt danh sách	quên `next = NULL`; dùng nút sau `free`	`gdb bt`; Valgrind `Invalid read`

gdb với struct (gdbdemo.c)

```c
typedef struct {
    int    id;
    char   name[12];
    double score;
} Student;

Student s = { 101, "An", 8.5 };
Student *p = &s;
```

- `print *p`: in toàn struct qua con trỏ.
- `ptype`: xem định nghĩa kiểu khi quên.
- Hiệu hai địa chỉ = `offsetof`: `score` ở byte 16 ($4 + 12$, không cần đệm).
- `set print pretty on`: mỗi trường một dòng.

```
$ gcc -g -O0 gdbdemo.c -o gdbdemo && gdb -q ./gdbdemo
(gdb) break 18
(gdb) run
(gdb) print s
$1 = {id = 101, name = "An\000\000\000\000\000\000\000\000\000\000",
  score = 8.5}
(gdb) print p->name
$2 = "An\000\000\000\000\000\000\000\000\000\000"
(gdb) ptype Student
type = struct {
    int id;
    char name[12];
    double score;
}
(gdb) print sizeof(s)
$3 = 24
(gdb) print (char *)&s.score - (char *)&s
$4 = 16
```

Định dạng thật của gdb 14; số thứ tự $n đã đánh lại cho gọn.

Tra cứu: man và cờ biên dịch của tuần

Lệnh	Nội dung
`man 3 qsort`	Chữ ký, hợp đồng comparator, "order ... is undefined"
`man 3 bsearch`	Tìm nhị phân tổng quát
`man 3 strcmp`	So sánh chuỗi (Bài 2)
`man 3 offsetof`	Vị trí byte của trường
`man 3 memcpy`	Chép khối byte (deep copy, `isort`)
`man 3 strdup`	Vì sao cần `_POSIX_C_SOURCE`
`man 3 scanf`	Tập ký tự `%[^\n]`, độ rộng tối đa

```
# chuan cua mon
gcc -Wall -Wextra -std=c11 -g bai04.c -o bai04
# bat tran so (comparator tru!)
gcc -g -fsanitize=undefined cmpbad.c -o cmpbad
# xem cho trinh bien dich chen dem
gcc -Wpadded -c layout.c
# kiem tra bo nho (Bai 2, 4, 5, 7)
valgrind --leak-check=full ./bai05 < tests/bai05.in
```

`-fsanitize=undefined` in ra dòng "`signed integer overflow`" ngay trong comparator – thay vì một thứ tự sai không ai để ý.

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Kế hoạch 90 phút và các điểm dừng

Thời lượng	Nội dung hướng dẫn	Sinh viên làm (điểm dừng)
0–8’	Mục tiêu; câu hỏi dẫn: `struct Node` 16 byte	Chạy `layout.c`
8–20’	Struct, khởi tạo, gán = sao chép, typedef; căn lề và đệm	**Bài 1**
20–30’	Con trỏ tới struct: `->`, `T *`, `const T *`	**Bài 3**
30–40’	Enum, union, union có nhãn	**Bài 6**
40–55’	Con trỏ hàm, `qsort`, hàm so sánh, tính ổn định	**Bài 4** (tới `qsort`)
55–70’	Bubble, selection, insertion; số đo thật	**Bài 2**: hàm sắp xếp
70–82’	Struct chứa con trỏ, shallow/deep copy, Valgrind	**Bài 5** + `valgrind`
82–88’	Danh sách liên kết với `List` (nối Lab 3)	Khung **Bài 7**
88–90’	Tổng kết, yêu cầu nộp bài	–
Sau 90’	Sinh viên **tự làm** hoàn chỉnh Bài 2, 4, 7; giảng viên hỗ trợ tại chỗ	

Quy tắc của buổi thực hành

Gõ tay, không copy–paste từ PDF. Mỗi bài: `gcc -Wall -Wextra -std=c11 -g` **sạch cảnh báo**; chạy ví dụ của đề *và* ít nhất một trường hợp biên; Bài 2, 4, 5, 7 thêm `valgrind --leak-check=full`, cả với input sai.

Bài 1 – Quản lý tọa độ robot

```c
typedef struct {
    int x;
    int y;
} Point;

/* 1 neu a + b khong vuot mien int */
static int add_ok(int a, int b) {
    return !((b > 0 && a > INT_MAX - b) ||
             (b < 0 && a < INT_MIN - b));
}

/* Cap nhat qua con tro; 0 neu thanh cong,
   -1 neu tran so (khi do *p giu nguyen) */
int point_move(Point *p, int dx, int dy) {
    if (!add_ok(p->x, dx) || !add_ok(p->y, dy))
        return -1;
    p->x += dx;
    p->y += dy;
    return 0;
}

void point_print(const Point *p) {
    printf("(%d,%d)\n", p->x, p->y);
}
```

```
Nhap toa do ban dau (x y): 3 4
Vi tri ban dau: (3,4)
Nhap dich chuyen (dx dy), Ctrl+D de ket thuc: 2 1
Vi tri sau khi cap nhat: (5,5)
Nhap dich chuyen (dx dy), Ctrl+D de ket thuc: ^D
```

Kiểm chứng: `(3,4) + (2,1)` → `(5,5)` (đề); `(0,0) + (-5,3) + (5,-3)` → `(0,0)`; `(INT_MAX,0) + (1,0)` → báo tràn; abc → lỗi, mã 1.

- **Giả định**: tọa độ nguyên (theo ví dụ), nhập `3 4`; dịch chuyển nhiều lần tới EOF.
- `point_move` nhận `Point *`: sửa vị trí *của robot*; `point_print` chỉ đọc: `const Point *`.
- Cách khác: `Point point_moved(Point p, ...)` trả struct mới.

Bài 2 – Quản lý sinh viên (1/3): kiểu dữ liệu và nhập

```c
#define NAME_LEN 50          /* toi da 49 ky tu + '\0' */
typedef struct {
    int    id;
    char   name[NAME_LEN];
    int    age;
    double score;
} Student;

/* Doc mot dong ho ten (co dau cach) */
static int read_name(char name[NAME_LEN]) {
    if (scanf(" %49[^\n]", name) != 1) {  /* 49 = NAME_LEN-1 */
        fprintf(stderr, "Loi: khong doc duoc ho ten\n");
        return -1;
    }
    int c = getchar();
    if (c != '\n' && c != EOF) {          /* ten qua dai */
        fprintf(stderr, "Loi: ho ten toi da %d ky tu\n",
                NAME_LEN - 1);
        return -1;
    }
    return 0;
}
/* "trong doan" thay vi "ngoai doan": NaN bi loai */
static int valid_score(double x) {
    return x >= MIN_SCORE && x <= MAX_SCORE;
}
```

- Mảng `Student` cấp phát động: `malloc((size_t)n * sizeof *list)` – mẫu của Lab 3, phần tử là struct.
- `" %49[^\n]"`: bỏ khoảng trắng đầu (cả `'\n'` còn sót sau %d), đọc tới hết dòng, tối đa 49 ký tự.
- Còn ký tự trước `'\n'` ⇒ tên quá dài: báo lỗi thay vì cắt ngầm.
- **Bẫy NaN**: `scanf("%lf")` nhận `nan`; `x < 0 || x > 10` đều sai với NaN nên NaN *lọt qua*. Viết điều kiện "trong đoạn".
- Mã SV phải dương và **không trùng** – nếu trùng, "tìm theo id" mất nghĩa.

Bài 2 – Quản lý sinh viên (2/3): tìm, cập nhật, thống kê

```c
/* Chi so cua SV co ma id, -1 neu khong co */
int find_index(const Student *list, int n, int id) {
    for (int i = 0; i < n; i++)
        if (list[i].id == id)
            return i;
    return -1;
}

double average_score(const Student *list, int n) {
    double sum = 0;                    /* n >= 1 */
    for (int i = 0; i < n; i++)
        sum += list[i].score;
    return sum / n;
}

/* trong handle_choice, case 3: */
i = find_index(list, n, id);
if (i < 0)
    printf("Khong tim thay sinh vien %d\n", id);
else
    list[i].score = score;             /* sua ban goc */

/* case 5: in TAT CA SV dat diem cao nhat */
double m = max_score(list, n);
for (i = 0; i < n; i++)
    if (list[i].score >= m - EPS)
        printf("  %d - %s\n", list[i].id, list[i].name);
```

```
[1]In [2]Tim [3]Sua diem [4]TB [5]Max ... : 2
Ma SV can tim: 103
Tim thay:
 Ma SV  Ho ten                    Tuoi   Diem
   103  Le Van Cuong                21   7.50
... : 3
Ma SV va diem moi: 104 9
Da cap nhat diem cua 104 thanh 9.00
... : 4
Diem trung binh ca lop: 8.25
... : 5
Diem cao nhat: 9.00
  102 - Tran Thi Binh
  104 - Pham Thi Dung
```

- `find_index` trả *chỉ số*: dùng chung cho "tìm" và "cập nhật", giữ được `const`.
- Hàm tính toán không `printf`; `handle_choice` điều phối menu.
- Hai bạn cùng 9.00: in *cả hai* (giả định).

Bài 2 – Quản lý sinh viên (3/3): sắp xếp bằng con trỏ hàm

```c
typedef int (*StudentCmp)(const Student *a,
                          const Student *b);

/* Sap xep chen, ON DINH */
void sort_students(Student *list, int n,
                   StudentCmp cmp) {
    for (int i = 1; i < n; i++) {
        Student key = list[i];
        int j = i - 1;
        while (j >= 0 && cmp(&list[j], &key) > 0) {
            list[j + 1] = list[j];
            j--;
        }
        list[j + 1] = key;
    }
}
int cmp_score_desc(const Student *a,
                   const Student *b) {
    if (a->score < b->score) return 1;
    if (a->score > b->score) return -1;
    return 0;
}
int cmp_name_asc(const Student *a,
                 const Student *b) {
    return strcmp(a->name, b->name);
}
```

```
... : 7
Sau khi sap xep theo ten:
 Ma SV  Ho ten                    Tuoi   Diem
   103  Le Van Cuong                21   7.50
   101  Nguyen Van An               20   7.50
   104  Pham Thi Dung               20   9.00
   102  Tran Thi Binh               19   9.00
... : 6
Sau khi sap xep theo diem giam dan:
 Ma SV  Ho ten                    Tuoi   Diem
   104  Pham Thi Dung               20   9.00
   102  Tran Thi Binh               19   9.00
   103  Le Van Cuong                21   7.50
   101  Nguyen Van An               20   7.50
```

- Một thuật toán, hai tiêu chí: chỉ đổi hàm so sánh.
- Xếp theo tên *rồi* theo điểm: cùng 9.00 thì Dung trước Binh – vẫn theo tên, nhờ insertion sort **ổn định**; `qsort` không bảo đảm điều này.

Bài 2 – các chỗ đề chưa nói rõ và lựa chọn của lời giải

"Sắp xếp theo tên": họ tên hay tên riêng?

Lời giải: `strcmp` trên *cả chuỗi* `name` (như hàm `byName` của tài liệu Lab) ⇒ thực chất xếp theo **họ**. Theo thói quen Việt Nam "theo tên" là theo *tên riêng* (từ cuối): `strrchr(name, ' ')` lấy từ cuối, so sánh nó trước, rồi so sánh cả chuỗi. Hai cách cho thứ tự khác nhau – nêu rõ trong chú thích.

Thuật toán sắp xếp

Đề không chỉ định. Lời giải tự viết insertion sort để luyện mục 5 của tài liệu (Bài 4 bắt buộc `qsort`). Dùng `qsort` cũng đúng đề.

Các giả định khác

- Điểm thang 10, kiểu `double`; tuổi 15–100; mã SV dương, không trùng; $1 \le n \le 1000$.
- Họ tên không dấu (ASCII), tối đa 49 ký tự; `strcmp` phân biệt hoa thường (`'Z' < 'a'`).
- Nhiều SV cùng điểm cao nhất: in tất cả.
- Giao diện: menu lặp tới khi chọn 0 hoặc hết dữ liệu.

Bẫy

`scanf("%s", s->name)` đọc "Nguyen" rồi để "Van An" cho lệnh `scanf("%d")` kế tiếp ⇒ nhập tuổi thất bại.

Bài 3 – Tài khoản ngân hàng

```c
typedef struct {
    int       id;
    long long balance;      /* so du, luon >= 0 */
} Account;

typedef enum {
    TX_OK, TX_BAD_AMOUNT, TX_INSUFFICIENT, TX_OVERFLOW
} TxResult;

/* Kiem tra TRUOC, sua SAU */
TxResult deposit(Account *acc, int amount) {
    if (amount <= 0)
        return TX_BAD_AMOUNT;
    if (acc->balance > LLONG_MAX - amount)
        return TX_OVERFLOW;
    acc->balance += amount;  /* sua ban goc */
    return TX_OK;
}

TxResult withdraw(Account *acc, int amount) {
    if (amount <= 0)
        return TX_BAD_AMOUNT;
    if (amount > acc->balance)
        return TX_INSUFFICIENT;
    acc->balance -= amount;
    return TX_OK;
}
```

```
Nhap ma tai khoan va so du ban dau: 1001 300000
Tai khoan 1001, so du: 300000
Lenh: n <tien> = nap, r <tien> = rut, q = thoat
> r 400000
So du khong du. Tai khoan 1001, so du: 300000
> n 0
So tien phai > 0. Tai khoan 1001, so du: 300000
> n 200000
Thanh cong. Tai khoan 1001, so du: 500000
> r 500000
Thanh cong. Tai khoan 1001, so du: 0
> q
```

- Chữ ký giữ đúng đề; đề không nói kiểu trả về $\Rightarrow$ trả `enum` cho biết *lý do* thất bại.
- `tx_message(r)`: `switch` không `default` để `-Wswitch` nhắc khi thêm mã mới.
- Rút đúng bằng số dư (về 0) là hợp lệ – ca biên.

Bài 3 – phân tích: từ Lab 3, Bài 14 đến Account

Cùng ý tưởng, gói lại

Lab 3: `bool withdraw(long long *balance, long long amount)`.
Lab 4: `TxResult withdraw(Account *acc, int amount)` – con trỏ tới *cả tài khoản*; thêm trường mới không phải đổi chữ ký.

- `balance` là `long long`: tổng nhiều lần nạp có thể vượt `int`; `amount` là `int` theo đề.
- Kiểm tra `amount <= 0` *trước*: quên thì `withdraw(&acc, -500)` *cộng* tiền.
- Kiểm tra tràn trước khi cộng: `balance > LLONG_MAX - amount`.

Bẫy: `scanf("%d")` với số quá lớn

Chuẩn C: số không biểu diễn được trong `int` là **hành vi không xác định**. Đo trên máy soạn: `n 99999999999` được đọc thành 2147483647 và giao dịch "thành công" – không có lỗi nào.

```
> n 99999999999
Thanh cong. Tai khoan 1, so du: 2147483747
```

Chương trình thật đọc chuỗi rồi dùng `strtol` (kiểm tra `errno == ERANGE`) – sẽ gặp lại ở Lab 5 khi đọc dữ liệu từ file.

Bài 4 – Dữ liệu cảm biến (1/2): enum, struct, nhập

```c
typedef enum {
    STATUS_NORMAL, STATUS_WARNING, STATUS_ERROR,
    STATUS_COUNT                    /* = 3 */
} SensorStatus;
typedef struct {
    float        temperature;    /* thu tu nhu de */
    int          timestamp;
    SensorStatus status;
} SensorData;                       /* sizeof = 12 */
static const char *const STATUS_NAMES[STATUS_COUNT] =
    { "NORMAL", "WARNING", "ERROR" };

int parse_status(const char *word, SensorStatus *out) {
    for (int s = 0; s < STATUS_COUNT; s++)
        if (strcmp(word, STATUS_NAMES[s]) == 0) {
            *out = (SensorStatus)s;
            return 0;
        }
    return -1;                      /* "OK" -> loi */
}
void count_by_status(const SensorData *a, int n,
                     int count[STATUS_COUNT]) {
    for (int s = 0; s < STATUS_COUNT; s++)
        count[s] = 0;
    for (int i = 0; i < n; i++)
        count[a[i].status]++;    /* enum lam chi so */
}
```

- Mỗi lần đo một dòng: `<thời điểm> <nhiệt độ> <NORMAL|WARNING|ERROR>`; trạng thái nhập bằng *chữ*, đổi sang enum bằng `parse_status`.
- `parse_status` chỉ trả giá trị trong miền $\Rightarrow$ `count[a[i].status]++` an toàn (cùng lập luận với Lab 3, Bài 20).
- Kiểm tra: thời điểm ≥ 0; nhiệt độ hữu hạn (`isfinite` chặn `nan`, `inf`) và $\geq -273{,}15$.
- Cộng dồn trung bình bằng `double` dù dữ liệu là `float`.

Bài 4 – Dữ liệu cảm biến (2/2): qsort theo thời gian

```c
int cmp_timestamp(const void *pa, const void *pb) {
    const SensorData *a = pa;
    const SensorData *b = pb;
    return (a->timestamp > b->timestamp)
         - (a->timestamp < b->timestamp);
}

qsort(a, (size_t)n, sizeof a[0], cmp_timestamp);
```

```
Nhiet do trung binh: 28.50
Nhiet do lon nhat  : 41.20
  NORMAL : 3 lan
  WARNING: 1 lan
  ERROR  : 1 lan

Sau khi sap xep theo thoi gian tang dan:
 Thoi diem  Nhiet do  Trang thai
         0     34.80  NORMAL
        60     35.00  NORMAL
       120     36.50  NORMAL
       180     41.20  WARNING
       240     -5.00  ERROR
```

Input (`tests/bai04.in`): `120 36.5 NORMAL`, `60 35.0 NORMAL`, `180 41.2 WARNING`, `0 34.8 NORMAL`, `240 -5.0 ERROR`.

- **Giả định**: trung bình và lớn nhất tính trên *mọi* lần đo như đề viết. Lần đo `ERROR` kéo trung bình từ 36,88 xuống **28,50** – hệ thống thật thường loại chúng ra: nêu trong báo cáo.
- Hai lần đo cùng thời điểm: `qsort` không giữ thứ tự nhập (đo trên máy soạn: `5 30 NORMAL`, `5 31 WARNING` ra ngược lại). Cần xác định ⇒ khoá phụ.
- `sizeof a[0]`: kích thước *phần tử*, tự đúng khi đổi kiểu.

Bài 5 – Struct với cấp phát động

```c
struct Student {
    char *name;
    int   age;
};

int student_init(struct Student *s, const char *name, int age) {
    size_t len = strlen(name) + 1;      /* +1 cho '\0' */
    s->name = malloc(len);
    if (s->name == NULL)
        return -1;
    memcpy(s->name, name, len);
    s->age = age;
    return 0;
}
/* Deep copy: ban sao CHUOI rieng */
int student_deep_copy(struct Student *dst,
                      const struct Student *src) {
    return student_init(dst, src->name, src->age);
}

void student_free(struct Student *s) {
    free(s->name);
    s->name = NULL;
}
```

```c
struct Student shallow = a;     /* chep CON TRO */
struct Student deep;
if (student_deep_copy(&deep, &a) != 0) { ... }
shallow.name[0] = '#';
shallow.age = 99;
student_free(&a);     /* chuoi chung a/shallow */
shallow.name = NULL;  /* KHONG free lan nua    */
student_free(&deep);
```

```
Ho ten: Nguyen Van An
Tuoi  : 20
sizeof(struct Student) = 16, strlen(name) + 1 = 14
shallow.name == a.name ? CO
deep.name    == a.name ? KHONG
Sau khi sua shallow.name[0] = '#' va shallow.age = 99:
a       : name = "#guyen Van An", age = 20
shallow: name = "#guyen Van An", age = 99
deep    : name = "Nguyen Van An", age = 20
```

Đã lược cột địa chỉ %p (đổi mỗi lần chạy).

age là *giá trị*: shallow có bản riêng (20/99); name là *con trỏ*: dùng chung. deep không bị ảnh hưởng.

Bài 5 – phân tích: so sánh shallow copy và deep copy

	Shallow copy (`b = a`)	**Deep copy** (`student_deep_copy`)
Chép gì	Mọi byte của struct: *địa chỉ* chuỗi	Struct *và* nội dung chuỗi (vùng mới)
Chi phí	16 byte, không `malloc`	`malloc` + chép `strlen + 1` byte; có thể thất bại
Sửa `b.name[0]`	`a.name` đổi theo	`a` không đổi
Giải phóng	**đúng một** `free` cho cả hai	mỗi bản một `free`
Khi nào dùng	chỉ đọc tạm, có *một* chủ sở hữu rõ ràng	bản sao cần sống độc lập

Giả định của lời giải

Một sinh viên (đề không nói số lượng); tên tối đa 255 ký tự, đọc vào bộ đệm rồi cấp *đúng* `strlen + 1` byte; tuổi 15–100.

Kiểm chứng

Valgrind: `All heap blocks were freed`; chương trình gọi đúng 2 `malloc`, 2 `free`. Thêm `student_free(&shallow)` ⇒ `Invalid free` (mục 8).

Bài 6 – Union lưu dữ liệu đo

```c
union Value {               /* DUNG CHUNG mot vung nho */
    int   i;
    float f;
};
typedef enum { KIND_INT = 1, KIND_FLOAT = 2 } Kind;
typedef struct {
    Kind        kind;       /* nhan: truong nao hop le */
    union Value v;
} Measurement;

/* trong read_measurement: */
m->kind = (Kind)choice;
k = (m->kind == KIND_INT) ? scanf("%d", &m->v.i)
                          : scanf("%f", &m->v.f);
if (k != 1 || !line_is_empty()) { /* "3.5" khi chon int */
    fprintf(stderr, "Loi: ... khong dung kieu ...\n");
    return -1;
}

void print_measurement(const Measurement *m) {
    switch (m->kind) {
    case KIND_INT:   printf("So nguyen: %d\n", m->v.i); break;
    case KIND_FLOAT: printf("So thuc  : %g\n", m->v.f); break;
    }
}
```

```
Kieu du lieu (1 = so nguyen, 2 = so thuc, 0 = thoat): 1
Nhap gia tri: 42
So nguyen: 42
Kieu du lieu (1 = so nguyen, 2 = so thuc, 0 = thoat): 2
Nhap gia tri: 3.14
So thuc  : 3.14
Kieu du lieu (1 = so nguyen, 2 = so thuc, 0 = thoat): 0

sizeof(int) = 4, sizeof(float) = 4
sizeof(union Value) = 4  (= truong lon nhat, khong phai tong)
sizeof(Measurement) = 8  (nhan 4 byte + union 4 byte)
Ghi u.f = 1.5 roi doc u.i: 1069547520 (0x3FC00000)
```

Nhận xét về `sizeof`

Union 4 byte = trường lớn nhất (`int`, `float` đều 4); struct có hai trường sẽ là 8. Nhưng union *không tự biết* trường nào hợp lệ – dòng cuối: đọc `i` sau khi ghi `f` cho số vô nghĩa. Thêm nhãn tốn 4 byte: `Measurement` = 8.

Bài 7 – Danh sách liên kết đơn

```c
struct Node *create_node(int value) {
    struct Node *n = malloc(sizeof *n);
    if (n == NULL)
        return NULL;
    n->data = value;
    n->next = NULL;            /* khong de rac */
    return n;
}

void list_print(const List *list) {
    for (const struct Node *p = list->head; p != NULL;
         p = p->next)
        printf("%d -> ", p->data);
    printf("NULL\n");
}

void list_free(List *list) {
    struct Node *p = list->head;
    while (p != NULL) {
        struct Node *next = p->next;  /* lay TRUOC free */
        free(p);
        p = next;
    }
    list->head = list->tail = NULL;
    list->size = 0;
}
/* main: while (scanf("%d", &v) == 1) list_append(&list, v); */
```

```
$ echo "10 20 30" | ./bai07
Nhap cac so nguyen (Ctrl+D de ket thuc):
Danh sach (3 nut): 10 -> 20 -> 30 -> NULL
$ ./bai07 < /dev/null
Nhap cac so nguyen (Ctrl+D de ket thuc):
Danh sach (0 nut): NULL
$ echo "1 2 x 3" | ./bai07
Nhap cac so nguyen (Ctrl+D de ket thuc):
Loi: du lieu khong phai so nguyen
```

- **Giả định**: "thêm node" = thêm vào *cuối* (giữ thứ tự nhập); nhập số nguyên tới EOF.
- Danh sách rỗng là ca biên: in `NULL`. Nhập sai giữa chừng: giải phóng các nút đã tạo rồi mới thoát.
- Mở rộng (đề cương: thêm, xoá, tìm): ghép `search`, `delete_value` của Lab 3 – nhớ cập nhật `tail`.

Kiểm chứng cả 7 bài bằng một script (kỹ năng Lab 1)

```
#!/bin/bash
# check.sh - bien dich, chay thu, kiem tra bo nho
mkdir -p out
for f in bai0*.c; do
    exe="${f%.c}"
    gcc -Wall -Wextra -std=c11 -g "$f" -o "$exe" \
        || { echo "LOI BIEN DICH: $f"; continue; }
    [ -f "tests/$exe.in" ] || continue
    valgrind -q --leak-check=full --error-exitcode=9 \
        "./$exe" < "tests/$exe.in" > "out/$exe.txt"
    echo "$exe: ma tra ve $?"
done
diff out/bai02.txt expected/bai02.txt && echo "Bai 2 khop"
```

- `tests/bai02.in`: 4 sinh viên + chuỗi lựa chọn `2 103 2 999 3 104 9 4 5 7 6 0` – phủ mọi chức năng, kể cả "không tìm thấy".
- Bài có menu càng cần file input: gõ tay mỗi lần sửa mã là không kiểm thử lại.
- Mã trả về 9 $\Rightarrow$ Valgrind thấy lỗi bộ nhớ; 1 $\Rightarrow$ chương trình tự báo lỗi nhập.
- Thêm một file input *sai* cho mỗi bài: nhánh lỗi cũng phải sạch Valgrind.

Nội dung

1. Mở đầu: từ Lab 3 sang Lab 4
2. Struct và typedef
3. Bố cục bộ nhớ: căn lề và đệm
4. Con trỏ tới cấu trúc
5. Enum và union
6. Con trỏ hàm và `qsort`
7. Thuật toán sắp xếp $O(n^2)$
8. Struct chứa con trỏ, danh sách liên kết
9. Lỗi thường gặp và gỡ lỗi
10. Bài tập thực hành
11. Tổng kết

Câu hỏi kiểm tra trong báo cáo

1. Vì sao `sizeof(struct Node)` $= 16$? Dùng gì để kiểm chứng?
2. Sắp lại các trường của `struct C` thế nào để còn 8 byte?
3. `struct Student b = a;` chép gì? Vì sao không so sánh `a == b`?
4. `p->x` và `(*p).x`; vì sao `*p.x` sai?
5. Khi nào truyền `T`, khi nào `T *`, khi nào `const T *`?
6. Gán `Day d = 42;` có cảnh báo không? Lúc nào gcc giúp được với enum?
7. `sizeof` của `union {char s[5]; int i;}`? Vì sao cần nhãn?
8. Đọc khai báo `int (*op)(int, int)` và `int *op(int, int)`.
9. Hàm so sánh của qsort nhận gì, trả gì? Vì sao không `return a - b`?
10. Sắp xếp *ổn định* là gì? Thuật toán nào trong ba thuật toán ổn định? qsort thì sao?
11. Vì sao insertion sort chỉ cần $n - 1$ phép so sánh với mảng đã sắp?
12. Shallow copy và deep copy khác nhau thế nào với `struct {char *name; int age;}`?

Tiêu chí chấm bài

Đúng đắn – Sạch cảnh báo – Tổ chức hàm – Bền vững (kiểm tra nhập, `malloc`, `free` cả ở nhánh lỗi) – Sạch Valgrind (Bài 2, 4, 5, 7) – **Mô hình dữ liệu** (kiểu phù hợp, `const` đúng, hàm so sánh đúng hợp đồng) – Hiểu biết (nêu giả định) – Kiểm chứng tại trường hợp biên.

Chuẩn bị cho tuần sau: File I/O cấp thấp (system call)

Đọc trước

Slide lý thuyết SP05 (*Low-level File I/O*): file descriptor, `open`/`read`/`write`/`close`, `errno`, `lseek`; `man 2 open`, `man 2 read`, `man 2 lseek`.

Luyện thêm nếu còn thời gian

- Bài 2: thêm lựa chọn "xếp theo tên riêng" (từ cuối của họ tên); viết lại menu bằng bảng con trỏ hàm.
- Bài 4: tính trung bình *không* gồm lần đo `ERROR`; thêm khoá phụ cho thời điểm trùng.
- Bài 7: thêm `list_find` và `list_remove` (cập nhật `tail`); kiểm tra bằng Valgrind.
- Chạy `sorts.c` trên máy lab: so số đo với bảng trong slide; thử `-O0` và `-O2`.

Câu hỏi dẫn sang tuần sau

Muốn lưu danh sách `Student` của Bài 2 để lần chạy sau đọc lại, ta có thể ghi thẳng mảng xuống file: `write(fd, list, n * sizeof *list)`. Bản ghi thứ k khi đó nằm ở byte nào của file – và vì sao file này có thể đọc sai trên một máy hoặc trình biên dịch khác? → **`read`/`write`/`lseek`** và bố cục byte của struct.

Cảm ơn đã lắng nghe!